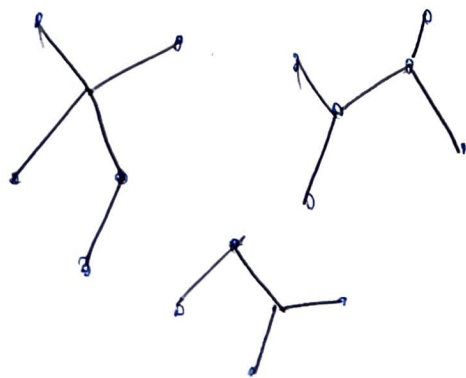


Union find data Structure:

①

In the last class we talked of Kruskal algorithm for computing minimum spanning trees. ~~And~~ In the algorithm there was an important step which actually dictates the running time of the algorithm and that step was how to detect if there is a cycle that gets formed when add an ~~edge~~ edge in the current set of edges which were already selected as part of the min spanning tree.

If you recall, we would try to maintain the collection of components. So we would like to look at a data structure called Union find data structure. So let us look at the Kruskal algorithm at some point of time. & It has picked a set of edges already which has formed a some sort of forest as follows



(2)

There should not be any cycle existing.

Now when the algorithm is trying to add a new edge, it needs to check if it forms a cycle or not.



Suppose there is the new edge being added. This edge forms a cycle, and

one way of detecting whenever certain edge would form a cycle is to check if its two end points lie in the same tree of this forest. A tree is same as the connected component and so it suffices to check if whether the two endpoints of an edge lie in the same connected component.

So, we have to somehow maintain our collection of connected component, so as the algorithm proceeds the no of connected components reduces by one with every steps.

Initially we started off with n connected components, each vertex being a component and eventually we will have only one connected components. This is how the algorithm proceeds.

we ~~now~~ are now going to

(3)

abstract this problem and capture it as a problem on maintaining a disjoint collection of disjoint sets.

So what's the setting now? I have a universe of elements, let us say

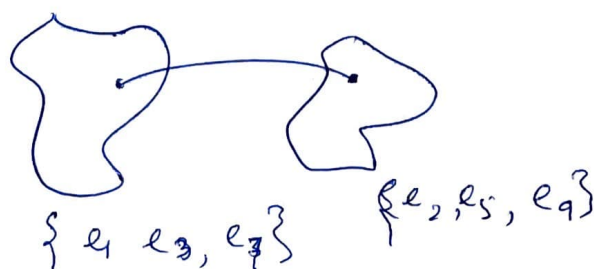
$$U = \{e_1, e_2, \dots, e_n\} \quad \begin{array}{l} n \text{ elements in} \\ \text{the universe} \end{array}$$

Initially each element $\{e_i\}$ is a set in itself like

$$\{e_1\} \quad \{e_2\} \quad \dots \quad \{e_n\}$$

These elements in the Kruskal algorithm corresponds to the vertices of the graph. So what are we going to do now is maintain a ~~set~~ collection of disjoint sets under the operation of (i) union and the other operation is (ii) find(u)

What does union do? Suppose we have two connected components and the edge that we add runs between these two connected components then the resulting thing would be one single connected component



So if the 1st component $\{e_1, e_3, e_7\}$ is in it and the 2nd component has $\{e_2, e_5, e_9\}$ then after union we should not have these 2 sets. They should get replaced by 1 set which is $\{e_1, e_2, e_3, e_5, e_7, e_9\}$. So Union takes two sets as parameters, as $\text{Union}(S_1, S_2)$.

The other operation $\text{find}(x)$ takes an element as input and tells which set it belongs to. There is a unique set in which x lies, because collection set is a partition always a partition of universe. So given the definition of $\text{find}(x)$ how do we check whether the addition of an edge ~~say~~ ~~is~~ from a cycle or not?

well, suppose the end points of the edge are x and y , we do $\text{find}(x)$ and $\text{find}(y)$ if they both return the same set, then we conclude that, it is forming a cycle, otherwise if the sets are different then it is not forming a cycle and the edge is added using $\text{Union}(\text{find}(x), \text{find}(y))$ operation.

So if we write down the Kruskal algorithm now, it would look like this, assuming these two operations are available.

1. $T = \emptyset$

2. Sort the edges in the increasing order of length
let $e_1, e_2, e_3, \dots, e_m$ be the ordering

3. for $i = 1$ to m do

 let $e_i = (u, v)$

 if $\text{find}(u) \neq \text{find}(v)$ then

 {
 $T = T \cup \{e\}$

 } $\text{Union}(\text{find}(u), \text{find}(v))$

4. return T .

So this would be the Kruskal algorithm looking like. We now need to understand what kind of data structure we need to use for maintaining the collection of disjoint set so that these two operations viz (i) union and (ii) find can be done very quickly.

Before we proceed further let us try to count how many times do we need the operation $\cup$ and how many times do we need the find operation

Union operation

every time we add an edge we do union.
how many edges can be there in the tree?

well, it's exactly $n-1$, so we will have exactly $n-1$ union.

no of find () .

For every edge we consider we have to do 2 find(). So in the worst how many edges we need to consider? all the edges.

So the number of find() that we need to use is $\leq$ the no of edges ~~is~~ m in the graph and it is $2m$.

The loop in the algorithm runs from 1 to m , but you know you can always break out of the loop the moment $n-1$ edges are included.

So what will be the total running time of the Kruskal algorithm.

Suppose a union takes U times and find takes F time then we have

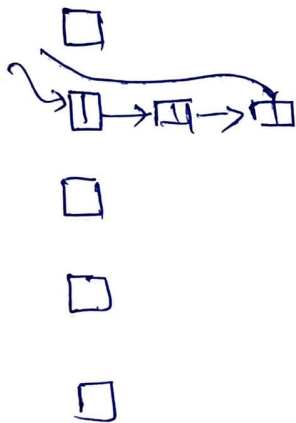
$$T = O(m \log m + Un + mF)$$

So we need to find a good datastructure, by good we mean which will have small U and F .

Now what are the possible data-structures that we can use to maintain the collection disjoint sets.

Linked list?

Initially we have as many linked list as there are sets. one linked list for each set.



Now let us see how much time does $union()$ and $find()$ take if we use linked list.

Union: Union takes constant time. Even if the linked list become large, we can keep

track of the front and rear end of each linked list as shown and

if we do that we can combine two linked list in constant time. So union will not take too much time?

$find()$: $find$ takes $O(\text{size of the linked list})$ which in the worst case could be $O(n)$.

8
If we do this we the running time of the Kruskal algorithm becomes $O(mn)$ as.

$$T = O(m \log m + n + mn)$$

this is too large. we are looking for a ~~data structure~~ ~~that should give~~ running time something like $n \log n$ for this two operations. So, if we are looking for something $n \log m$, The quantity U and F should not be more than $\log m$. So linked list is not a good data structure.

So which other data structure we can think of?

A tree? A heap? no we will have a new data structure and what will a new data structure be? I will just show this thing to you and then you will understand what is happening

Suppose my universe is

$$U = \{a, b, c, d, e, f\} \text{ six elements}$$

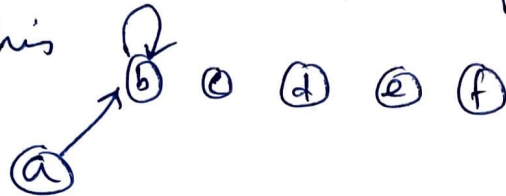
Initially the sets in my collection is

(a) (b) (c) (d) (e) (f) all singleton

sets, we have one node for each of these six sets

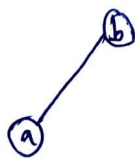
Now suppose we do

$\text{Union}(\text{find}(a), \text{find}(b))$. Each node have a data field and a reference field. So what will happen is we will make one of this guys point to the other like this



Suppose now we do $\text{Union}(\text{find}(a), \text{find}(b))$

So when we say $\text{find}(a)$, we will start from a and keep going up till we reach the root, in this case b is the root. How do we know it is the root? well its pointer points to itself. we are actually having parent pointer with each node. So for each set it is represented by one of the elements in the set. In some sense all the elements of a set elect a leader, a for example in the set below a , and

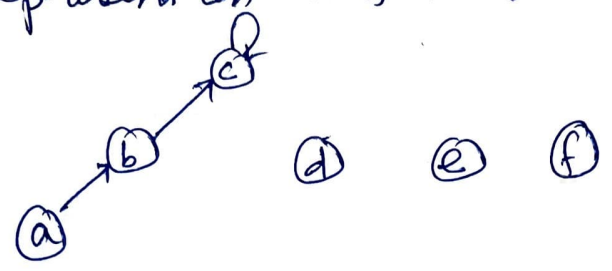


b are the two elements and b is the leader.

So when we say $\text{find}(a)$ it return to me reference to the leader b .

So what union does is does is, it takes the root of the two trees and makes one point to the other.

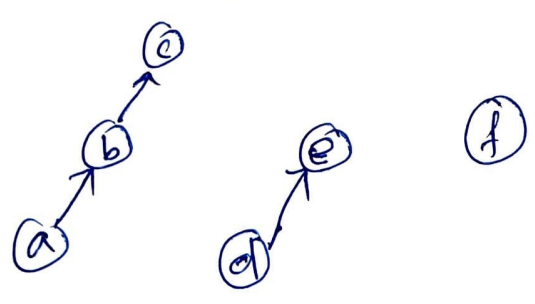
So for instance if we do union ($\text{find}(a)$, $\text{find}(c)$) we might decide to make b point to c and in our representation this looks like



So when now when I say $\text{find}(a)$ or $\text{find}(b)$ or even $\text{find}(c)$ it returns c or reference to c. It is best to return reference because that can be used by union operation to merge the trees.

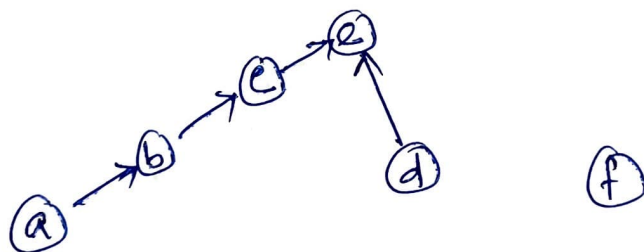
Suppose now we union ($\text{find}(d)$, $\text{find}(e)$)

so what will we do? we will link up the roots for d and e, so lets say we decide to make d point to e. as follows



Now If we do an operation like

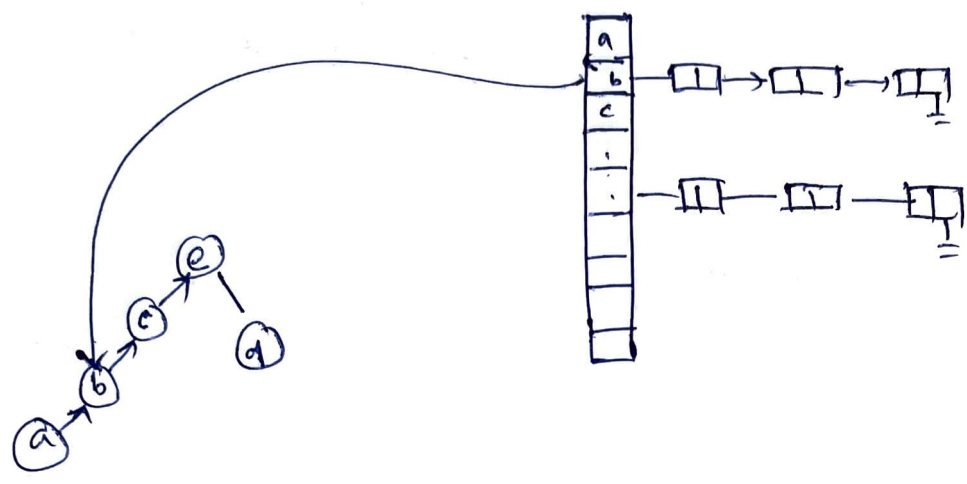
$\text{union}(\text{find}(a), \text{find}(d))$ then what does $\text{find}(a)$ return? It returns a reference/pointer to c , $\text{find}(d)$ returns a pointer to e and we need to link up these two roots. We can make c point to e or we can make e point to c whichever we please. Let's say we decide to c points to e . Then ~~for~~ what we get is the following and of course



f would be sitting on its own. So I hope you understand what find does. It starts from the element and keep tracing up the pointers up till it hits the root.

Now the question arises how do we go to the element when I say $\text{find}(a)$ let's say. Well recall that we have a

data structure for graph may be adjacency matrix or adjacency list. Suppose we have a adjacency list representation



So for each vertex in the adjacency list we have another reference to the tree as shown above, so that we can access a particular node in the tree straight away.

So what is the problem with this implementation? Let us how much time does both union and find operation take?

union: Union takes as input reference of two root nodes and all it has to do is to modify one pointer to point to the other. So union takes $O(1)$ time.

find()

! how much time does find take? It can take a lot of time. because it might have to travel a long way to reach the root. In the ~~construct~~ worst case it might take $O(n)$ time

Eg: If we decide to merge first a and b, then a and c, then a and d, then a and e, and a and f. in this order and this thing would go ~~set~~ really bad.

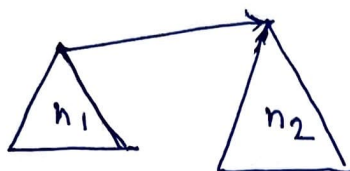
So we have to do the union in more cleaner manner.

So recall that when we were trying linking the two roots we had the option of linking the first one to the 2nd or the 2nd one to the first one. We will now exploit that.

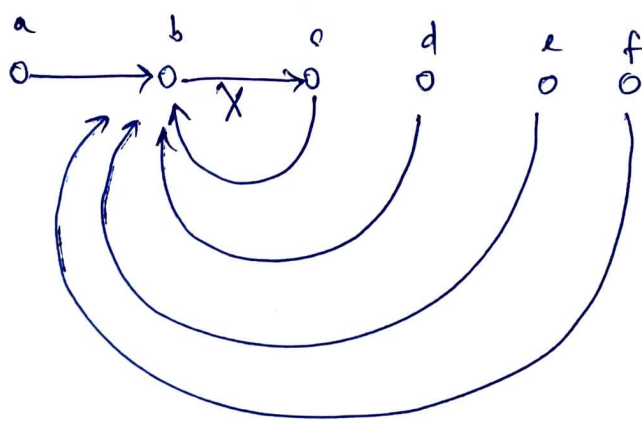
We are going to use a rule called

Union by rank: This rule says that suppose we have two trees with say n_1 and n_2 nodes in them, then we merge the

lighter tree point to the heavier tree. How do we qualify ~~the~~ a tree to be lighter or heavier? well, by their no of nodes. Suppose without loss of generality that $n_1 < n_2$ we will move tree



having n_1 node point to the ~~base~~ root of the tree having n_2 nodes. Now we will not have this kind of a scenario in which for example we have these 6 elements, all of them form a chain. Rather it will be looking like the following



So now what will be the height of this

tree? 1 only. and so find will take very little time.

(15)

So we now need to see if we use this rule what could be the height of the tree in the worst case. How high can the tree become? How will you argue that this rule of union by rank will lead to trees of height almost $\log n$. So the claim that we are going to make is this

Claim: A tree with n_1 nodes have height $\leq \lg n_1$

How do we prove this claim?

So let us use induction to prove this

we are combining two trees one with n_1 nodes and the other with n_2 nodes



w.l.o.g assume
 $n_1 \leq n_2$

Let the induction hypothesis is true till this stage. That means



Now we have to show that as a consequence of this we will get a new tree with $(n_1 + n_2)$ nodes and whose height $\leq \log(n_1 + n_2)$

lets see if that is true

what will be the height of new tree?

~~the height could be~~

So height of the resultant tree would either be the height of the tree with n_2 nodes or height of the tree with n_1 nodes + 1.

ie

$$\text{height of the new tree} = \max(h_2, h_1 + 1)$$

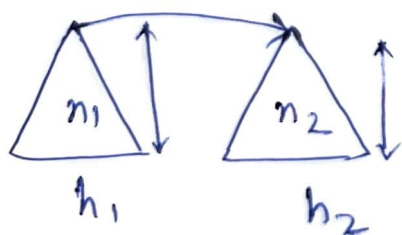
if this equals h_2 then

$$h_2 \leq \log n_2 \leq \log(n_1 + n_2)$$

if this equals $h_1 + 1$

$$\begin{aligned} \text{then } h_1 + 1 &\leq \log n_1 + 1 \\ &= \log 2n_1 \\ &\leq \log(n_1 + n_2) \end{aligned}$$

because $n_2 \geq n_1$



Now is the base case true?

Base Case

for $n=1$ the height becomes 0.

So let us define the height. The tree with only one node is of height 0.

if $n=2$ the height becomes 1.



height 0 tree



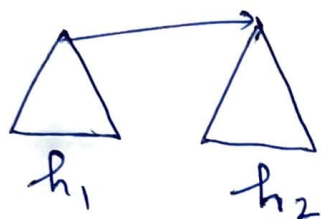
height 1 tree. So in the

height we are counting the edges, on the longest path from the leaves to the root.

It is to be noted that induction is on the number of nodes in the tree. So we are assuming the induction hypothesis to be true for ~~certain no~~ of nodes less than certain no of nodes. and when we have the trees having this no of nodes we get a tree with larger no of nodes and the statement is ~~going to hold~~ continuing to be held true. So this is all about union by rank. Rank actually means the number of nodes in the tree.

We can also do a union by height, we can make the shallow tree point to the tree with larger height. This approach will also work. Let's see why

What are we doing now? ~~when~~ We have two trees with height h_1 and h_2 respectively with $h_1 < h_2$



what we do in this case is we ~~attach~~ ^{make} the root of the tree with height h_1 point to the root of the tree with height h_2 .

The claim that we are going to make now is this

claim: The tree of height h has at least 2^h nodes.

This is just the converse of the earlier claim there we were trying to prove a tree with so many nodes is of height ^{almost} $\log(\text{no of nodes})$ and here we

we are trying to prove if the height is h then it has atleast 2^h no of nodes. which will antinomially means the tree can never be of height more than $\log n$, because if the height of the tree is more than $\log n$ will have more than n no of nodes which is not possible as there are only n nodes in the graph. to begin with. So that will place a $\log n$ bound on the height of the tree.

Now how will we prove the claim? ~~once~~ well, once again by induction.

hypothesis: suppose the claim is true for all trees of height up to a certain no. and when we merge the two trees the resulting tree has height $h = \max(h_2, h_1 + 1)$
 h is the height of the new tree. ^{↑ happens when $h_1 = h_2$}

no of nodes in the new tree is

$$\begin{aligned} &= n_1 + n_2 \\ &\geq 2^{h_1} + 2^{h_2} \\ &\geq 2^{h_2} \\ &\geq 2^{h_1} + 2^{h_1} \quad \because h_1 < h_2 \end{aligned} \quad \left| \begin{array}{l} n_1 \geq 2^{h_1} \\ n_2 \geq 2^{h_2} \end{array} \right.$$

$$\geq 2^{h_i+1}$$

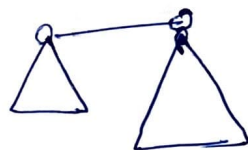
$$\begin{aligned} \text{So the no of nodes} &\geq \max(2^{h_2}, 2^{h_i+1}) \\ &= \max 2^{\max(h_2, h_i+1)} \\ &= 2^{h_i} \end{aligned}$$

So proof are very similar, we are just turning them ~~around~~ around. So both of these schemes can be used to do the unions. So given this how much time does unions take?

Union : constant time

$$O(1)$$

~~we~~ we just need update in the reference



to achieve this we need keep track of the no of nodes or height of the tree in the root node and this information

is not very difficult to maintain. If we do union by rank we just add the no of nodes in both the tree and keep it in the root node of the larger tree. On the other hand if we do the union by height we need to update the root of the ~~highest~~ higher tree with $\max(h_2, h_i+1)$.

Now how much time does find take?

find: takes no more than $O(\log n)$ time.

So the total time taken by Kruskal algorithm then becomes

$$\begin{aligned} T &= O(m \log m + n + m \cdot \log m) \\ &\approx O(m \log m) \approx O(m \log n) \end{aligned}$$

what is

$\log m$? what could be

the ~~max~~ maximum value of m ?
well n^2

So max value of $\log m \leq 2 \log n$

and minimum value of m in connected graph $\log(n)$

So

$$\log_2 n \leq \log m < 2 \log n$$

$$\text{So } \log m = \Theta(\log n)$$